

答案就在 `dcms_mcu_config_uds.h` 的静态配置表中。

区分机制：编译期静态配置表，非运行时判断

`DCMS_MCU_UDS_TOPIC_CONFIG` 表通过四个互斥的标志位来定义每个端点的角色：

```
#define DCMS_MCU_UDS_TOPIC_CONFIG \
{ \
    /*TOPIC ID                TOPIC_NAME                PUB_EN  SUB_EN CLIENT SERVER*/ \
    /* topic 订阅者 */ \
    {0, DCMS_SUB_EOLCALIB_RESULT_RTOS,  "/sys/eol_result_rtos",      0u, 1u, 0u, 0u, ...}, \
    {0, DCMS_CALIB_DATA_ACK_RTOS,      "/sys/send_stedelta",      0u, 1u, 0u, 0u, ...}, \
    {0, DCMS_TOPIC_DIAG_AVM_GESTURE,    "/sys/diag_avm_gesture/v1", 0u, 1u, 0u, 0u, ...}, \
    /* topic 发布者 */ \
    {0, DCMS_TOPIC_HMI_CALIB_REQ,       "/sys/hmi_calib/request/v1", 1u, 0u, 0u, 0u, ...}, \
    /* service client (请求方) */ \
    {0, DCMS_CLIENT_DMM_CALIB_MODE_CMD, "/sys/dmm_calib/v1",      0u, 0u, 1u, 0u, ...}, \
    {0, DCMS_CLIENT_START_EOLCALIB_RTOS, "/sys/start_calib_rtos",  0u, 0u, 1u, 0u, ...}, \
    {0, DCMS_CLINET_GETCALIB_PARAM_RTOS, "/sys/inquire_parameter...", 0u, 0u, 1u, 0u, ...}, \
    {0, DCMS_TOPIC_CALIB_PRECONDITION_CHECK, "/sys/calib_precondition_check", 0u, 0u, 1u, 0u, ...}, \
    /* service server (响应方) */ \
    {0, DCMS_TOPIC_AVM_STATE_INFO,      "/sys/avm_state_info/v1",  0u, 0u, 0u, 1u, ...}, \
}
```

关键是：同一份配置表分别编译进 SOC 和 MCU 两侧固件，但两侧各自编译时，同一个 topic ID 的四个标志位可以不同。SOC 侧编译时 `DCMS_TOPIC_CALIB_PRECONDITION_CHECK` 是 `CLIENT=1`，MCU 侧编译时同一个 topic 是 `SUB_EN=1`（或 `SERVER=1`），DCMS 框架在初始化时读取配置表，静态确定每个端点的角色。

角色决定了 API 行为

角色	发送方 API	接收方 API	有应答?
PUB (发布者)	<code>dcms_mcu_topic_send()</code>	<code>dcms_mcu_topic_setup_callback()</code>	✗ 无
SUB (订阅者) (不发送)		<code>dcms_mcu_topic_setup_callback()</code>	✗ 无
CLIENT (客户端)	<code>dcms_mcu_topic_send()</code> + <code>service_client_setup_resp_callback()</code>	(对端处理)	✔ 有
SERVER (服务端) (对端发送)		<code>dcms_mcu_topic_setup_callback()</code> 收到请求 + <code>service_server_set_ackdata()</code> 回复	✔ 有

具体以标前置条件检查为例：

SOC 侧配置 `DCMS_TOPIC_CALIB_PRECONDITION_CHECK` -- `CLIENT=1`

发送: `dcms_mcu_topic_send(DCMS_TOPIC_CALIB_PRECONDITION_CHECK, &req)`
注册应答回调: `dcms_mcu_service_client_setup_resp_callback(..., DiagCalib_GetPreconditionDataCallback)`
：框架知道这个 topic 是 CLIENT 角色，发送后会将对端的 ack 路由到这个回调

MCU 侧配置 同一个 topic -- `SUB_EN=1`（或 `SERVER=1`）

注册接收回调: `dcms_mcu_topic_setup_callback(DCMS_TOPIC_CALIB_PRECONDITION_CHECK, DiagCalib_PreconditionCheckCallbackAck)`
：收到数据后触发回调，执行车辆信号检查
回复: `dcms_mcu_service_server_set_ackdata(DCMS_TOPIC_CALIB_PRECONDITION_CHECK, &respResult)`
：框架根据配置表知道这个 topic 期待回复，将数据路由回 CLIENT 侧

总结

server 侧不需要“判断”是 topic 还是 service。DCMS 框架在初始化时读取 `DCMS_MCU_UDS_TOPIC_CONFIG` 配置表，每个 topic ID 的角色（PUB/SUB/CLIENT/SERVER）就已经确定了。框架内部根据角色自动决定：

- SUB -- 只收不发，不期待应答
- SERVER -- 收到请求后，调用 `service_server_set_ackdata` 时框架自动将数据路由回对应的 CLIENT

命名约定（`DCMS_CLIENT_*`、`DCMS_SUB_*`）只是辅助可读性，真正的行为由配置表中的四个标志位列决定。